CREACIONALES

- Builder.py

main.py

```
class CoffeeBuilder:
    def reset(self):
        return self

    def set_size(self, size):
        self._size = size
        return self

    def add_milk(self, milk_type):
        self._milk = milk_type
        return self

    def add_sugar(self, spoons):
        self._sugar = spoons
        return self

    def add_extra(self, extra):
        self._extras.append(extra)
        return self

    def build(self):
        # Es importante pasar una copia de la lista (self._extras[:])
        # para que el siguiente café no comparta la misma lista de extras.
        coffee = Coffee(self._size, self._milk, self._sugar, self._extras[:])
        self.reset()
        return coffee


# Paso 3. Usarlo:
builder = CoffeeBuilder()

latte = (builder
         .set_size("grande")
         .add_milk("almendra")
         .add_sugar(2)
         .add_extra("canela")
         .build())

espresso = builder.set_size("pequeño").build()

print(latte)
print(espresso)
```

Console | I/O | AI Agent — 147 ms

```
Coffee(size=grande, milk=almendra, sugar=2, extras=
['canela'])
Coffee(size=pequeño, milk=None, sugar=0, extras=[])
```

-Prototype.py – Clonar objetos

main.py

```
import copy

# Paso 1. Creamos la clase con método clone:
class PromoBanner:
    def __init__(self, title, text, discount):
        self.title = title
        self.text = text
        self.discount = discount

    def clone(self):
        return copy.deepcopy(self)

    def __str__(self):
        return f"[{self.title}] {self.text} ({self.discount}% off)"


# Paso 2. Creamos un “prototipo” y lo clonamos:
base_promo = PromoBanner(
    title="Promo de la semana",
    text="Café + croissant",
    discount=20
)

# Clonamos y personalizamos para el lunes
promo_lunes = base_promo.clone()
promo_lunes.title = "Lunes de energía"

# Clonamos y personalizamos para el viernes
promo_viernes = base_promo.clone()
promo_viernes.discount = 30

# Mostramos los resultados
print(base_promo)
print(promo_lunes)
print(promo_viernes)
```

Console | I/O | AI Agent

```
[Promo de la semana] Café + croissant (20% off)
[Lunes de energía] Café + croissant (20% off)
[Promo de la semana] Café + croissant (30% off)
```

-Singleton.py – Una sola instancia

main.py

```python
# Paso 1. Creamos la clase:
class AppConfig:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if not cls._instance:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self, db_url=None):
        if not hasattr(self, "db_url"):
            self.db_url = db_url or "sqlite://local.db"


# Paso 2. La usamos:
config1 = AppConfig("postgres://prod")
config2 = AppConfig("otro")  # no cambia, sigue siendo el primero

print(config1.db_url)       # postgres://prod
print(config2.db_url)       # postgres://prod
print(config1 is config2)   # True
```

Console | I/O | AI Agent | 138 ms

```
postgres://prod
postgres://prod
True
```

- factory .py Fábrica de objetos

main.py

```python
# Paso 1. Crear clases por tipo de chocolate:
class Chocolate:
    def prepare(self):
        raise NotImplementedError


class ChocolateBar(Chocolate):
    def prepare(self):
        return "Preparando una barra de chocolate Wonka"


class FizzyLiftingDrink(Chocolate):
    def prepare(self):
        return "Preparando un Fizzy Lifting Drink (bebida burbujeante)"


class EverlastingGobstopper(Chocolate):
    def prepare(self):
        return "Preparando un Everlasting Gobstopper (caramelo eterno)"


# Paso 2. La “Fábrica” Wonka con el método Factory:
class WonkaFactory:
    def create_candy(self, candy_type: str) -> Chocolate:
        candy_type = candy_type.lower()

        if candy_type == "bar":
            return ChocolateBar()
        elif candy_type == "fizzy":
            return FizzyLiftingDrink()
        elif candy_type == "gobstopper":
            return EverlastingGobstopper()
        else:
            raise ValueError("Ese dulce no existe en la fábrica de Willy Wonka.")


# Paso 3. Usarla:
factory = WonkaFactory()

print(factory.create_candy("bar").prepare())
print(factory.create_candy("fizzy").prepare())
print(factory.create_candy("gobstopper").prepare())
```

Console | I/O | AI Agent | 132 ms

```
Preparando una barra de chocolate Wonka
Preparando un Fizzy Lifting Drink (bebida burbujeante)
Preparando un Everlasting Gobstopper (caramelo eterno)
```

ESTRUCTURALES

adapter .py

main.py

```python
# Paso 1. Definimos la interfaz objetivo y el sistema legacy:
# Interfaz que espera nuestra aplicación nueva
class PaymentProcessor:
    def pay(self, amount: float) -> str:
        raise NotImplementedError


# Sistema legacy (no lo podemos cambiar)
class LegacyPaymentGateway:
    def make_payment(self, value_in_cents: int) -> str:
        return f"Pago legacy procesado por {value_in_cents} centavos"


# Paso 2. Creamos el Adapter:
class LegacyPaymentAdapter(PaymentProcessor):
    def __init__(self, legacy_gateway: LegacyPaymentGateway):
        self.legacy = legacy_gateway

    def pay(self, amount: float) -> str:
        # Adapta la unidad (dólares -> centavos) y el nombre del método
        cents = round(amount * 100)
        return self.legacy.make_payment(cents)


# Paso 3. Usarlo:
gateway = LegacyPaymentGateway()
processor = LegacyPaymentAdapter(gateway)

print(processor.pay(25.50))  # Pago legacy procesado por 2550 centavos
```

Console / I/O / AI Agent

```
Pago legacy procesado por 2550 centavos
```

-decorator .py Añadir funcionalidades dinámicamente

main.py

```python
# Paso 1. Componente base y notificación concreta:
class Notifier:
    def send(self, message: str) -> None:
        raise NotImplementedError


class EmailNotifier(Notifier):
    def send(self, message: str) -> None:
        print(f"[Email] {message}")


# Paso 2. Decoradores que envuelven a otro Notifier:
class NotifierDecorator(Notifier):
    def __init__(self, wrapped: Notifier):
        self._wrapped = wrapped

    def send(self, message: str) -> None:
        self._wrapped.send(message)


class SMSDecorator(NotifierDecorator):
    def send(self, message: str) -> None:
        super().send(message)
        print(f"[SMS] {message}")


class SlackDecorator(NotifierDecorator):
    def send(self, message: str) -> None:
        super().send(message)
        print(f"[Slack] {message}")


# Paso 3. Componer en tiempo de ejecución:
notifier = SlackDecorator(SMSDecorator(EmailNotifier()))
notifier.send("Servidor caído")
```

Console / I/O / AI Agent

```
[Email] Servidor caído
[SMS] Servidor caído
[Slack] Servidor caído
```

-Facade .py – Una fachada simple sobre un subsistema complejo

main.py

```python
# Paso 1. El subsistema (varias clases independientes):
class Projector:
    def on(self):
        print("Proyector encendido")

    def off(self):
        print("Proyector apagado")


class SoundSystem:
    def set_volume(self, level):
        print(f"Volumen en {level}")

    def off(self):
        print("Sonido apagado")


class StreamingPlayer:
    def play(self, movie):
        print(f"Reproduciendo '{movie}'")

    def stop(self):
        print("Reproducción detenida")


# Paso 2. La fachada que lo orquesta todo:
class HomeTheaterFacade:
    def __init__(self, projector, sound, player):
        self.projector = projector
        self.sound = sound
        self.player = player

    def ver_pelicula(self, movie):
        self.projector.on()
        self.sound.set_volume(7)
        self.player.play(movie)

    def terminar(self):
        self.player.stop()
        self.sound.off()
        self.projector.off()
```

Console | I/O | AI Agent

```
Proyector encendido
Volumen en 7
Reproduciendo 'Interstellar'
Reproducción detenida
Sonido apagado
Proyector apagado
```

-Composite .py– Tratar objetos individuales y grupos de la misma forma

main.py

```python
# Paso 1. Componente común (hojas y compuestos comparten interfaz):
class FileSystemItem:
    def __init__(self, name: str):
        self.name = name

    def get_size(self) -> int:
        raise NotImplementedError


# Paso 2. Hoja (File) y Compuesto (Folder):
class File(FileSystemItem):
    def __init__(self, name: str, size: int):
        super().__init__(name)
        self._size = size

    def get_size(self) -> int:
        return self._size


class Folder(FileSystemItem):
    def __init__(self, name: str):
        super().__init__(name)
        self.children: list[FileSystemItem] = []

    def add(self, item: FileSystemItem):
        self.children.append(item)

    def get_size(self) -> int:
        return sum(child.get_size() for child in self.children)


# Paso 3. Construir el árbol y consultarlo:
root = Folder("proyecto")
root.add(File("README.md", 2))

src = Folder("src")
src.add(File("main.py", 10))
src.add(File("utils.py", 5))

root.add(src)
print(root.get_size())  # 17 (se trata igual un archivo que una carpeta)
```

Console | I/O | AI Agent | 149 ms

```
17
```

COMPORTAMIENTO

Strategy .py – Algoritmos intercambiables

main.py

```python
#Paso 1. Estrategias intercambiables:
class DiscountStrategy:
    def apply(self, total: float) -> float:
        raise NotImplementedError
class NoDiscount(DiscountStrategy):
    def apply(self, total): return total
class StudentDiscount(DiscountStrategy):
    def apply(self, total): return total * 0.90
class EmployeeDiscount(DiscountStrategy):
    def apply(self, total): return total * 0.80
class VIPDiscount(DiscountStrategy):
    def apply(self, total): return total * 0.70
#Paso 2. Contexto que recibe la estrategia:
class Cart:
    def __init__(self, strategy: DiscountStrategy):
        self.items = []
        self.strategy = strategy
    def add(self, price: float):
        self.items.append(price)
    def total(self) -> float:
        return self.strategy.apply(sum(self.items))
#Paso 3. Cambiar la estrategia sin tocar el carrito:
cart = Cart(StudentDiscount())
cart.add(100); cart.add(50)
print(cart.total()) # 135.0
cart.strategy = VIPDiscount()
print(cart.total()) # 105.0
```

Console | I/O | AI Agent

```
135.0
105.0
```

Observer .py – Notificar a varios suscriptores

main.py

```python
#Paso 1. Sujeto (observable) y observador:
class Subscriber:
    def update(self, video: str):
        raise NotImplementedError
class Channel:
    def __init__(self, name: str):
        self.name = name
        self._subs: list[Subscriber] = []
    def subscribe(self, sub: Subscriber):
        self._subs.append(sub)
    def unsubscribe(self, sub: Subscriber):
        self._subs.remove(sub)
    def publish(self, video: str):
        print(f"\n{self.name} publicó: {video}")
        for sub in self._subs:
            sub.update(video)
#Paso 2. Suscriptores concretos:s
class EmailSubscriber(Subscriber):
    def __init__(self, email): self.email = email
    def update(self, video):
        print(f" -> Email a {self.email}: nuevo video '{video}'")
class MobileSubscriber(Subscriber):
    def __init__(self, user): self.user = user
    def update(self, video):
        print(f" -> Push a {self.user}: '{video}'")
#Paso 3. Usarlo:
channel = Channel("DevTalks")
channel.subscribe(EmailSubscriber("ana@x.com"))
channel.subscribe(MobileSubscriber("@carlos"))
channel.publish("Patrones de Diseño en Python")
```

Console | I/O | AI Agent | 157 ms

```
DevTalks publicó: Patrones de Diseño en Python
 -> Email a ana@x.com: nuevo video 'Patrones de Diseño en Python'
 -> Push a @carlos: 'Patrones de Diseño en Python'
```

Command .py – Encapsular una petición como objeto

main.py

```python
#Paso 1. Receptor y comandos:
class Light:
    def __init__(self): self.is_on = False
    def turn_on(self): self.is_on = True; print("Luz: ON")
    def turn_off(self): self.is_on = False; print("Luz: OFF")
class Command:
    def execute(self): raise NotImplementedError
    def undo(self): raise NotImplementedError
class TurnOnCommand(Command):
    def __init__(self, light): self.light = light
    def execute(self): self.light.turn_on()
    def undo(self): self.light.turn_off()
class TurnOffCommand(Command):
    def __init__(self, light): self.light = light
    def execute(self): self.light.turn_off()
    def undo(self): self.light.turn_on()
#Paso 2. Invocador (el control remoto) con historial:
class RemoteControl:
    def __init__(self): self.history: list[Command] = []
    def press(self, command: Command):
        command.execute()
        self.history.append(command)
    def undo(self):
        if self.history:
            self.history.pop().undo()
#Paso 3. Usarlo:
light = Light()
remote = RemoteControl()
remote.press(TurnOnCommand(light)) # Luz: ON
remote.press(TurnOffCommand(light)) # Luz: OFF
remote.undo() # Luz: ON
remote.undo() # Luz: OFF
```

Console | I/O | AI Agent

```
Luz: ON
Luz: OFF
Luz: ON
Luz: OFF
```

Template .py– Esqueleto de un algoritmo

main.py

```python
# Paso 1. Clase base con el algoritmo completo:
class Report:
 # Template method: define el orden, no se sobreescribe
    def generate(self, data: dict) -> str:
        parts = [self.header(data), self.body(data), self.footer(data)]
        return "\n".join(parts)
    def header(self, data): raise NotImplementedError
    def body(self, data): raise NotImplementedError
    def footer(self, data): raise NotImplementedError
#Paso 2. Subclases que rellenan los “huecos”:
class TextReport(Report):
    def header(self, data): return f"=== {data['title']} ==="
    def body(self, data): return data['content']
    def footer(self, data): return f"-- Generado por {data['author']} --"
class HTMLReport(Report):
    def header(self, data): return f"<h1>{data['title']}</h1>"
    def body(self, data): return f"<p>{data['content']}</p>"
    def footer(self, data): return f"<footer>{data['author']}</footer>"
#Paso 3. Usarlo:
data = {"title": "Ventas Q4", "content": "Crecimos 12%", "author": "A. Acosta"}
print(TextReport().generate(data))
print()
print(HTMLReport().generate(data))
```

Console | I/O | AI Agent

```
=== Ventas Q4 ===
Crecimos 12%
-- Generado por A. Acosta --

<h1>Ventas Q4</h1>
<p>Crecimos 12%</p>
<footer>A. Acosta</footer>
```

State .py – Cambiar de comportamiento según el estado interno

main.py

```python
#Paso 1. Una clase por estado:
class OrderState:
    def pay(self, order): print("Acción no permitida en este estado")
    def ship(self, order): print("Acción no permitida en este estado")
    def deliver(self, order):print("Acción no permitida en este estado")
class Pending(OrderState):
    def pay(self, order):
        print("Pago recibido")
        order.state = Paid()
class Paid(OrderState):
    def ship(self, order):
        print("Pedido enviado")
        order.state = Shipped()
class Shipped(OrderState):
    def deliver(self, order):
        print("Pedido entregado")
        order.state = Delivered()
class Delivered(OrderState):
    pass # estado final
#Paso 2. Contexto que delega en el estado:
class Order:
    def __init__(self):
        self.state: OrderState = Pending()
    def pay(self): self.state.pay(self)
    def ship(self): self.state.ship(self)
    def deliver(self): self.state.deliver(self)
#Paso 3. Usarlo:
order = Order()
order.ship() # Acción no permitida en este estado (aún está Pending)
order.pay() # Pago recibido
order.ship() # Pedido enviado
order.deliver() # Pedido entregado
```

Console | I/O | AI Agent

```
Acción no permitida en este estado
Pago recibido
Pedido enviado
Pedido entregado
```